Projet S3

Dossier de Base de Données

Gabriel Delpha | Janith Mapitiyage |
Rémy Stinus

2025-2026

Responsable : Nicolas Férey

université PARIS-SACLAY | IUT D'ORSAY

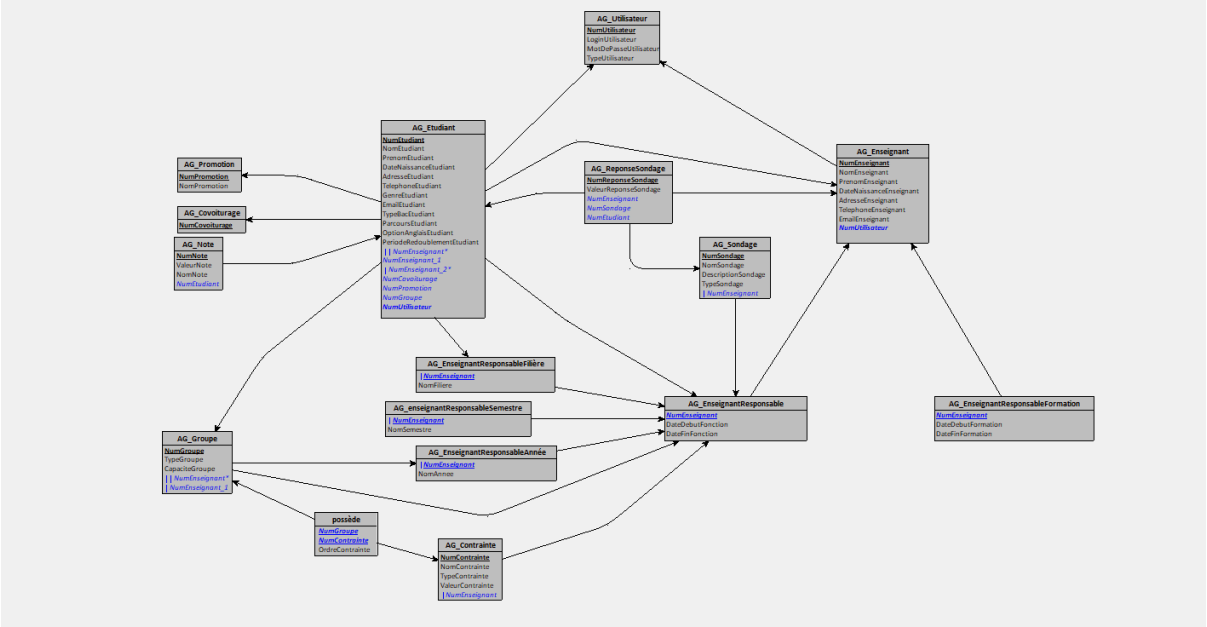
Table des Matières

1- Partage des tâches : 3
2 - Modèle Relationnel : 3
Différences Rendu ANALYSE : 3
3 - Descriptions 4
3.1 - Création des Tables : 4
3.2 - Insertion de données : 4
3.3 - Vues : 4
3.4 - Requêtes : 5
SELECT : 5
UPDATE : 6
INSERT : 6
DELETE : 6
3.5 - Procédures : 6
3.6 - Triggers : 8
3.7 - Fonctions : 9
AIDE : 9
FONCTIONS : 11

1- Partage des tâches :

- Modèle Relationnel : Gabriel
- Création de tables : Remy
- création des Tables/ Insertion des données : Gabriel / Remy
- Requêtes / vues : Janith
- Triggers / Procédures / Fonctions : Gabriel / Remy / Janith

2 - Modèle Relationnel :



Différences Rendu ANALYSE :

- Association entre AG_Etudiant et AG_Enseignant passe de n,n à 1.n car nous estimons désormais qu'un étudiant est ajouté par un unique enseignant.

- Association d'héritage AG_EnseignantResponsable et ses trois entités filles passe de S à XT car nous estimons désormais qu'un enseignant responsable est forcément soit responsable de filière de Semestre ou d'années.

- Ajout de l'entité AG_Utilisateur qui stock le LoginUtilisateur et le MotDePasseUtilisateur ainsi que le TypeUtilisateur (étudiant ou enseignant). Cette entité possède deux entités filles AG_Enseignant et AG_Etudiant.
  Nous ne l'avions pas fait avant car nuos pensions que les login et mdp seraient gérer par la base de données de l'iut mais nous avons vite compris que c'est nécessaire d'implémenter ceci afin de gérer les utilisateurs correctement.

3 - Descriptions

3.1 - Création des Tables :

Le script de création des tables est dans le fichier nommé AG_ScriptCreaBD.sql.

Nous avons généré en grande partie le script de création des tables grâce à Looping avec notre MCD. Nous l'avons peaufiné afin qu'il corresponde exactement à ce que nous voulions.

3.2 - Insertion de données :

Le script de création des tables est dans le fichier nommé AG_ScriptInsertBD.sql.

Nous avons généré le script d'insertion de données dans notre base de données grâce à Gemini. Le script d'insertion de données a été généré par rapport aux différentes contraintes d'attributs de notre script de création de tables.

3.3 - Vues :

Les Descriptions ci-dessous suivent l'ordre des Vues dans le fichier AG_ScriptVue.sql.

- Vue_Consulter_Reponses_Sondage : Vue permettant de consulter les réponses des sondages.

- Vue_Consulter_Etudiant_Note : Vue permettant aux étudiants de consulter leur notes.

- Vue_Consulter_Etudiant_Groupe : Vue permettant aux étudiants de consulter leur groupe (ne peut fonctionner que si l'étudiant à un groupe).

- Vue_Consulter_Etudiant_NoteGroupe : Vue permettant de consulter les notes des étudiants d'un groupe.

- Vue_Consulter_Groupe_Contraintes : Vue permettant de consulter les contraintes d'un groupe.

- Vue_Consulter_Enseignant : Vue permettant de consulter les informations d'un enseignant.

- Vue_Consulter_ListePromotion_Public : Vue permettant de consulter la liste des promotions accessibles publiquement.

- Vue_Consulter_ListePromotion_Privé : Vue permettant de consulter la liste des promotions pas encore accessibles publiquement.

- Vue_Consulter_Covoiturage : Vue permettant de consulter les covoiturage

- Vue_Consulter_Donnees_Pedagogiques : Vue permettant de consulter les infos pédagogiques des étudiants.

- Vue_Consulter_Etudiant_Sans_Groupe : Vue permettant de consulter les étudiants n'ayant pas encore de groupe.

- Vue_Consulter_ExportResponsable : Vue permettant de consulter les informations exportables.

3.4 - Requêtes :

Les requêtes expliquées ci-dessous sont les requêtes les plus fréquentes, dans l'utilisation des fonctionnalités de notre projet.

Les Descriptions ci-dessous suivent l'ordre des requêtes dans le fichier AG_ScriptRequete.sql.

SELECT :

- PREMIER SELECT : On cherche les étudiants présents dans le groupe numéro 4.

- DEUXIÈME SELECT : On cherche les étudiants ayant l'option Anglophone et covoiturage.

- TROISIÈME SELECT : On cherche la réponse sur le sondage QUALITE de l'étudiant Pierre Sounouvou.

- QUATRIÈME SELECT : On cherche les responsables des filières.

- CINQUIÈME SELECT : On cherche les contraintes du groupe numéro 5.

- SIXIÈME SELECT : On cherche la liste des étudiants du groupe numéro 3.

- SEPTIÈME SELECT : On cherche la liste des étudiants qui n'ont pas de groupe.

- HUITIÈME SELECT : On cherche les données privées de l'étudiant Matthieu Charroin.

- NEUVIÈME SELECT : On cherche les notes en mathématiques des étudiants du groupe numéro 1.

- DIXIÈME SELECT : On cherche tout le dossier pédagogique de l'étudiant Martin Lucas.

- ONZIÈME SELECT : Consulter les étudiants qui covoiturent avec l'étudiant numéro 15.

UPDATE :

- PREMIER UPDATE : L'enseignant ajoute l'étudiant numéro 4 dans le groupe numéro 5.

- DEUXIÈME UPDATE : L'enseignant retire l'étudiant 4 de son groupe

- TROISIÈME UPDATE : L'étudiant 10 rejoint le 50 groupe du covoiturage

INSERT :

- PREMIER INSERT : L'étudiant 10 valide le sondage

- DEUXIÈME INSERT : Création d'un covoiturage

- TROISIÈME INSERT : Ajout d'un étudiant

DELETE :

- PREMIER DELETE : L'enseignant supprime l'étudiant 4

3.5 - Procédures :

Les descriptions des procédures ci-dessous suivent l'ordre des procédures du fichier AG_ScriptProcedure.sql.

- ajouterContrainte : procédure pour ajouter une contrainte avec toute ses données en toute sécurité et avec respect des contraintes d'attributs.

- ajouterEtudiant : procédure pour ajouter un étudiant avec toute ses données en toute sécurité et avec respect des contraintes d'attributs.

- ajouterSondage : procédure pour ajouter un sondage avec toute ses données en toute sécurité et avec respect des contraintes d'attributs.

- modifierContrainte : procédure pour modifier une contrainte avec toute ses données en toute sécurité et avec respect des contraintes d'attributs.

- modifierEtudiant : procédure pour modifier un étudiant avec toute ses données en toute sécurité et avec respect des contraintes d'attributs.

- modifierSondage : procédure pour modifier un sondage avec toute ses données en toute sécurité et avec respect des contraintes d'attributs.

- supprimerContrainte : procédure pour supprimer une contrainte avec toute ses données en toute sécurité et avec respect des contraintes d'attributs.

- supprimerEtudiant : procédure pour supprimer un étudiant avec toute ses données en toute sécurité et avec respect des contraintes d'attributs.

- supprimerSondage : procédure pour supprimer un sondage avec toute ses données en toute sécurité et avec respect des contraintes d'attributs.

- affecterRoleEnseignantResponsableFormation : procédure pour affecter un à enseignant le rôle d'enseignant responsable de formation avec toute ses données en toute sécurité et avec respect des contraintes d'attributs.

- supprimerRoleEnseignantResponsableFormation : procédure pour supprimer un à enseignant le rôle d'enseignant responsable de formation avec toute ses données en toute sécurité et avec respect des contraintes d'attributs.

- modifierInfoEnseignantResponsableFormation : procédure pour modifier les informations d'un enseignant ayant le rôle d'enseignant responsable de formation avec toute ses données en toute sécurité et avec respect des contraintes d'attributs.

- ajouterNote : procédure pour ajouter une note avec toute ses données en toute sécurité et avec respect des contraintes d'attributs.

- modifiernote : procédure pour modifier une note avec toute ses données en toute sécurité et avec respect des contraintes d'attributs.
-
- ajouterEnseignant : procédure pour ajouter un enseignant avec toute ses données en toute sécurité et avec respect des contraintes d'attributs.

- modifierEnseignant : procédure pour modifier un enseignant avec toute ses données en toute sécurité et avec respect des contraintes d'attributs.

- supprimerEnseignant : procédure pour supprimer un enseignant avec toute ses données en toute sécurité et avec respect des contraintes d'attributs.

- supprimerCovoiturage : procédure pour supprimer un covoiturage avec toute ses données en toute sécurité et avec respect des contraintes d'attributs.

- ajouterCovoiturage : procédure pour ajouter un covoiturage avec toute ses données en toute sécurité et avec respect des contraintes d'attributs.

- affecterCovoiturageEtudiant : procédure pour affecter un covoiturage à un étudiant avec toute ses données en toute sécurité et avec respect des contraintes d'attributs.

- affecterGroupeEtudiant : procédure pour affecter un groupe à un étudiant avec toute ses données en toute sécurité et avec respect des contraintes d'attributs.

- affecterPromotionetudiant : procédure pour affecter une promotion à un étudiant avec toute ses données en toute sécurité et avec respect des contraintes d'attributs.

3.6 - Triggers :

Les descriptions des triggers ci-dessous suivent l'ordre des triggers du fichier AG_ScriptTrigger.sql.

- tr_check_groupe_before_insert : trigger pour vérifier l'insertion des données de groupe dans la table AG_Etudiant.

- tr_check_groupe_before_update : trigger pour vérifier l'update des données de groupe dans la table AG_Etudiant.

- tr_check_promotion_before_insert : trigger pour vérifier l'insertion des données de promotion dans la table AG_Etudiant.

- tr_check_promotion_before_update : trigger pour vérifier l'update des données de promotion dans la table AG_Etudiant.

- tr_check_covoiturage_before_insert : trigger pour vérifier l'insertion des données de covoiturage dans la table AG_Etudiant.

- tr_check_covoiturage_before_update : trigger pour vérifier l'update des données de covoiturage dans la table AG_Etudiant.

- tr_check_etudiant_before_insert : trigger pour vérifier l'insertion des données personnel de l'étudiant dans la table AG_Etudiant.

- tr_check_etudiant_before_update : trigger pour vérifier l'update des données personnel de l'étudiant dans la table AG_Etudiant.

- tr_check_utilisateur_etu_before_insert : trigger pour vérifier l'insertion des données d'utilisateur dans la table AG_Etudiant.

- tr_check_utilisateur_etu_before_update : trigger pour vérifier l'update des données d'utilisateur dans la table AG_Etudiant.

- tr_check_utilisateur_ens_before_insert : trigger pour vérifier l'insertion des données d'utilisateur dans la table AG_Enseignant.

- tr_check_utilisateur_ens_before_update : trigger pour vérifier l'update des données d'utilisateur dans la table AG_Enseignant.

- tr_check_note_before_insert : trigger pour vérifier l'insertion des données de note dans la table AG_Note.

- tr_check_note_before_update : trigger pour vérifier l'update des données de note dans la table AG_Note.

- tr_check_contrainte_before_insert : trigger pour vérifier l'insertion des données dans la table AG_Constrainte.

- tr_check_contrainte_before_update : trigger pour vérifier l'update des données de contrainte dans la table AG_Constrainte.

- tr_check_promotion_before_insert : trigger pour vérifier l'insertion des données de contrainte dans la table AG_Promotion.

- tr_check_promotion_before_update : trigger pour vérifier l'update des données de promotion dans la table AG_Promotion.

- tr_check_sondage_before_insert : trigger pour vérifier l'insertion des données de sondage dans la table AG_Sondage.

- tr_check_sondage_before_update : trigger pour vérifier l'update des données de sondage dans la table AG_Sondage.

3.7 - Fonctions :

AIDE :

Nous avons eu une erreur lors de la création des fonctions sur l'app MySQL que nous avons utiliser quand nous n'avions pas accès à phpMyAdmin, nous avons donc rajouté deux "paramètres" supplémentaire lors de l'initialisation des fonctions (DETERMINISTIC et READS SQL DATA).
Voici le **prompt gémini** qui nous donner la réponse à notre erreur :

```
DELIMITER $$

DROP FUNCTION IF EXISTS fn_etudiantExiste$$

CREATE FUNCTION fn_etudiantExiste(p_num INT)
RETURNS BOOLEAN
BEGIN
   DECLARE v_existe INT DEFAULT 0;

   SELECT COUNT(*) INTO v_existe
```

```
    FROM AG_Etudiant
    WHERE numEtudiant = p_num;

    RETURN (v_existe > 0);
END$$

DELIMITER ;
```

This function has none of DETERMINISTIC, NO SQL, or READS SQL DATA in its declaration…

Comment je corrige cette erreur ?

Pour corriger cela, il suffit d'ajouter READS SQL DATA (parce que tu fais un SELECT) et DETERMINISTIC (parce que le résultat ne change pas de manière aléatoire) juste avant le BEGIN.

Voici le code corrigé :

SQL

```
DELIMITER $$

DROP FUNCTION IF EXISTS fn_etudiantExiste$$

CREATE FUNCTION fn_etudiantExiste(p_num INT)
RETURNS BOOLEAN
DETERMINISTIC    -- Indique que le résultat est prévisible pour un même p_num
READS SQL DATA   -- Indique que la fonction contient des requêtes SELECT
BEGIN
    DECLARE v_existe INT DEFAULT 0;

    SELECT COUNT(*) INTO v_existe
    FROM AG_Etudiant
    WHERE numEtudiant = p_num;

    RETURN (v_existe > 0);
END$$

DELIMITER ;
```

Pourquoi ces mots-clés règlent le problème ?
En MySQL, quand le "Binary Logging" est activé (ce qui est souvent le cas), le serveur doit savoir si une fonction est "sûre" pour être répliquée sur un autre serveur.

DETERMINISTIC : Rassure MySQL en disant : "Si j'appelle cette fonction deux fois avec l'ID 5, j'aurai le même résultat".

READS SQL DATA : Prévient que tu vas lire dans les tables.

Fin du prompt gémini.

FONCTIONS :

Les descriptions ci-dessous suivent l'ordre des fonctions du fichier AG_ScriptFonction.sql.

- fn_etudiantExiste : Fonction pour vérifier si un étudiant existe dans la base de données.
- fn_enseignantExiste : Fonction pour vérifier si un enseignant existe dans la base de données.
- fn_groupeExiste : Fonction pour vérifier si un groupe existe dans la base de données.
- fn_promotionExiste : Fonction pour vérifier si une promotion existe dans la base de données.
- fn_covoiturageExiste : Fonction pour vérifier si un covoiturage existe dans la base de données.
- fn_sondageExiste : Fonction pour vérifier si un sondage existe dans la base de données.
- fn_utilisateurExiste : Fonction pour vérifier si un utilisateur existe dans la base de données.
- fn_estResponsableFormation : Fonction pour vérifier si un utilisateur existe dans la base de données.
- fn_estResponsableSemestre : Fonction pour vérifier si un enseignant est responsable de semestre.
- fn_estResponsableAnnee : Fonction pour vérifier si un enseignant est responsable d'année.
- fn_estResponsableFiliere : Fonction pour vérifier si un enseignant est responsable de filière.
- fn_getNbEtudiantsDansGroupe : Fonction pour récupérer le nombre d'étudiants dans un groupe.
- fn_groupeEstPlein : Fonction pour vérifier si un groupe est complet.
- fn_covoiturageEstPlein : Fonction pour vérifier si un covoiturage est complet.
- fn_contrainteExiste : Fonction pour vérifier si une contrainte existe dans la base de données.
- fn_estResponsable : Fonction pour vérifier si un enseignant est responsable.
- fn_noteExiste : Fonction pour vérifier si une note existe dans la base de données.